

Supervised learning

- Overview
- Notation
 - Model
 - Loss function
 - Training
 - Testing
- 1D Linear regression example
 - Model
 - Loss function
 - Training
 - Testing
- Where are we going?

Supervised learning

- Overview
- Notation
 - Model
 - Loss function
 - Training
 - Testing
- 1D Linear regression example
 - Model
 - Loss function
 - Training
 - Testing
- Where are we going?

Supervised learning overview

- **Supervised learning model** = mapping from one or more inputs to one or more outputs
- Model is a mathematical equation
- Computing the outputs from the inputs = **inference**
- Example:
 - Input is age and milage of secondhand Toyota Prius
 - Output is estimated price of car

Supervised learning overview

- **Supervised learning model** = mapping from one or more inputs to one or more outputs
- Model is a family of equations
- Computing the inputs from the outputs = **inference**
- Model also includes **parameters**
- Parameters affect outcome of equation
- **Training** a model = finding parameters that predict outputs “well” from inputs for a **training dataset** of input/output pairs

Supervised learning

- Overview
- Notation
 - Model
 - Loss function
 - Training
 - Testing
- 1D Linear regression example
 - Model
 - Loss function
 - Training
 - Testing
- Where are we going?

Notation:

- Input:

$\mathbf{x}$

Variables always Roman letters

- Output:

y

Normal = scalar

Bold = vector

Capital Bold = matrix

- Model: $y = \mathbf{f}[\mathbf{x}]$

Functions always square brackets

Normal = returns scalar

Bold = returns vector

Capital Bold = returns matrix

Notation example:

- Input:

$$\mathbf{x} = \begin{bmatrix} \text{age} \\ \text{mileage} \end{bmatrix}$$



Structured or
tabular data

- Output:

$$y = [\text{price}]$$

- Model: $y = f[\mathbf{x}]$

Model

- Parameters:

- Model : ϕ

$$y = f[x, \phi]$$

← Parameters always
Greek letters

Loss function

- Training dataset of I pairs of input/output examples:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^I$$

- **Loss function** or **cost function** measures how bad model is:

$$L[\phi, f[\mathbf{x}_i, \phi], \{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^I]$$

or for short:

$$L[\phi]$$

← Returns a scalar that is smaller when model maps inputs to outputs better

Training

- Loss function:

$L[\phi]$  Returns a scalar that is smaller when model maps inputs to outputs better

- Find the parameters that minimize the loss:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} [L[\phi]]$$

Testing

- To test the model, run on a separate **test dataset** of input / output pairs
- See how well it **generalizes** to new data

Supervised learning

- Overview
- Notation
 - Model
 - Loss function
 - Training
 - Testing
- 1D Linear regression example
 - Model
 - Loss function
 - Training
 - Testing
- Where are we going?

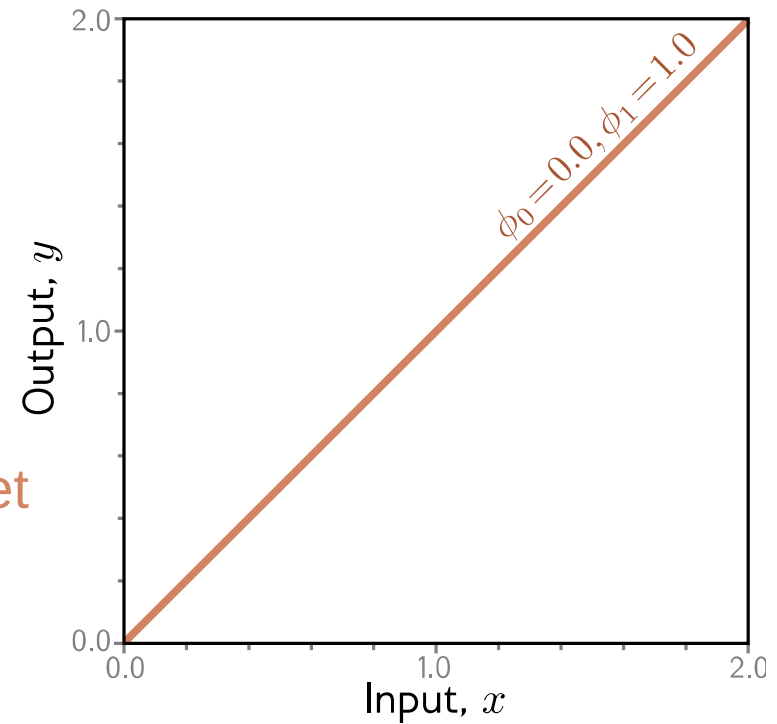
Example: 1D Linear regression model

- Model:

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x \end{aligned}$$

- Parameters

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix} \begin{array}{l} \leftarrow \text{y-offset} \\ \leftarrow \text{slope} \end{array}$$



Example: 1D Linear regression model

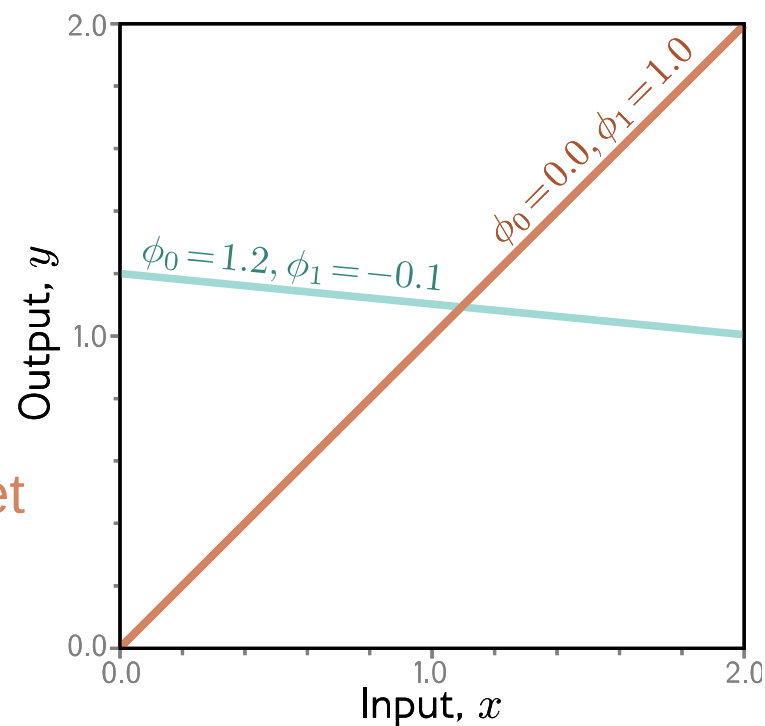
- Model:

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x \end{aligned}$$

- Parameters

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix}$$

← y-offset
← slope



Example: 1D Linear regression model

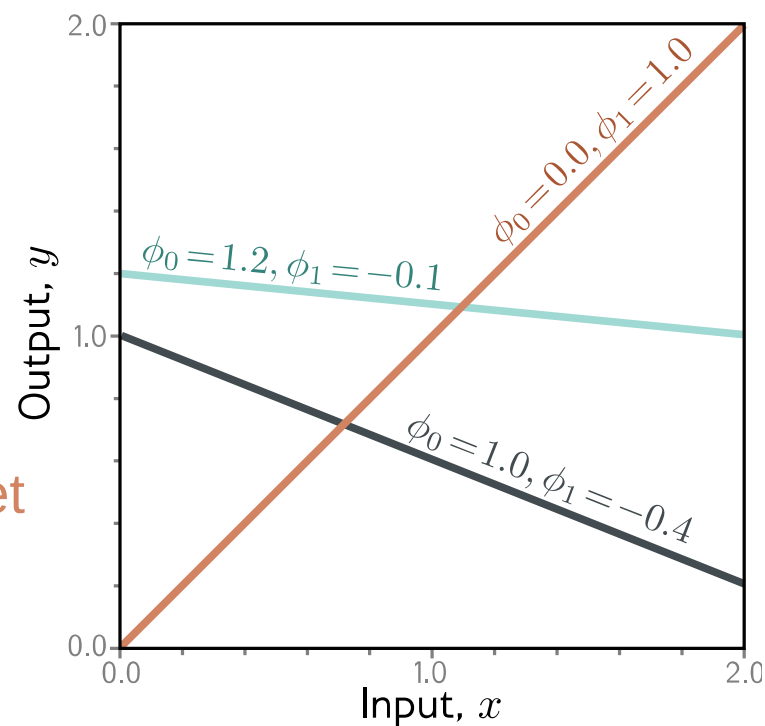
- Model:

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x \end{aligned}$$

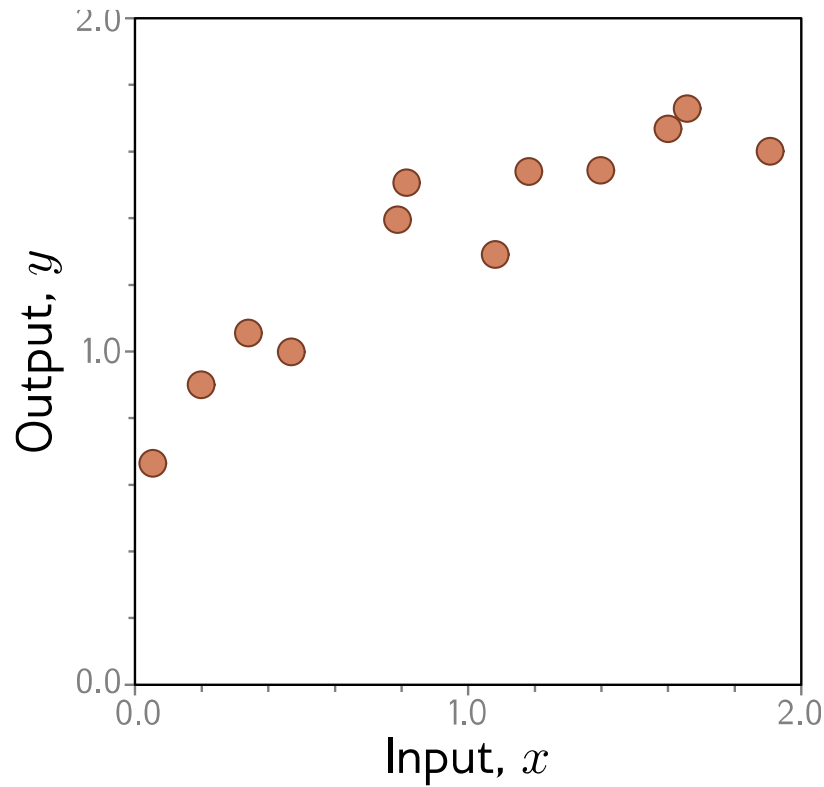
- Parameters

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix}$$

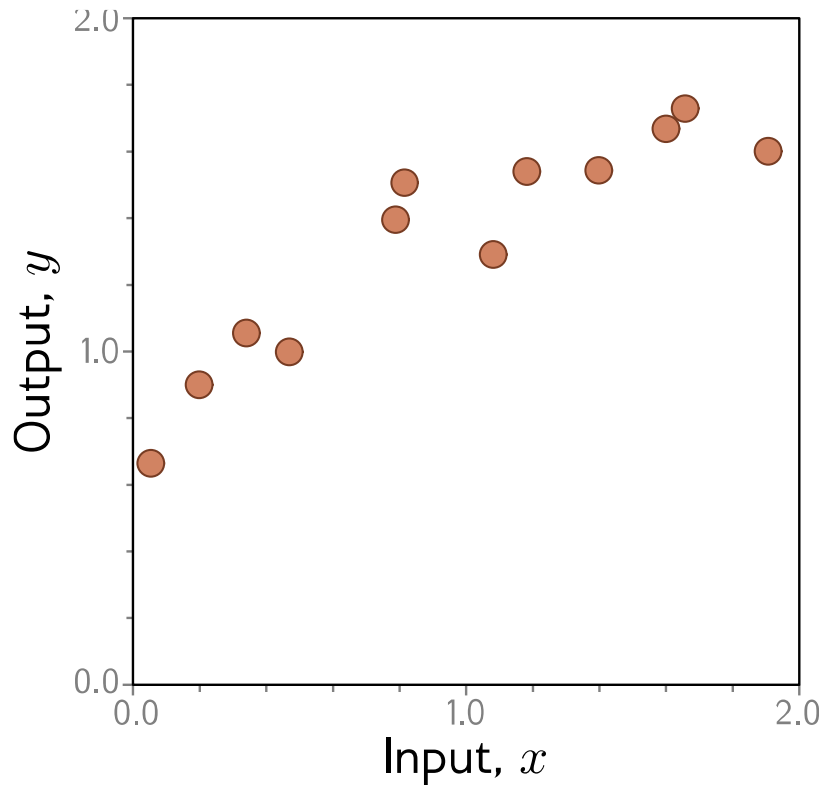
← y-offset
← slope



Example: 1D Linear regression training data



Example: 1D Linear regression training data

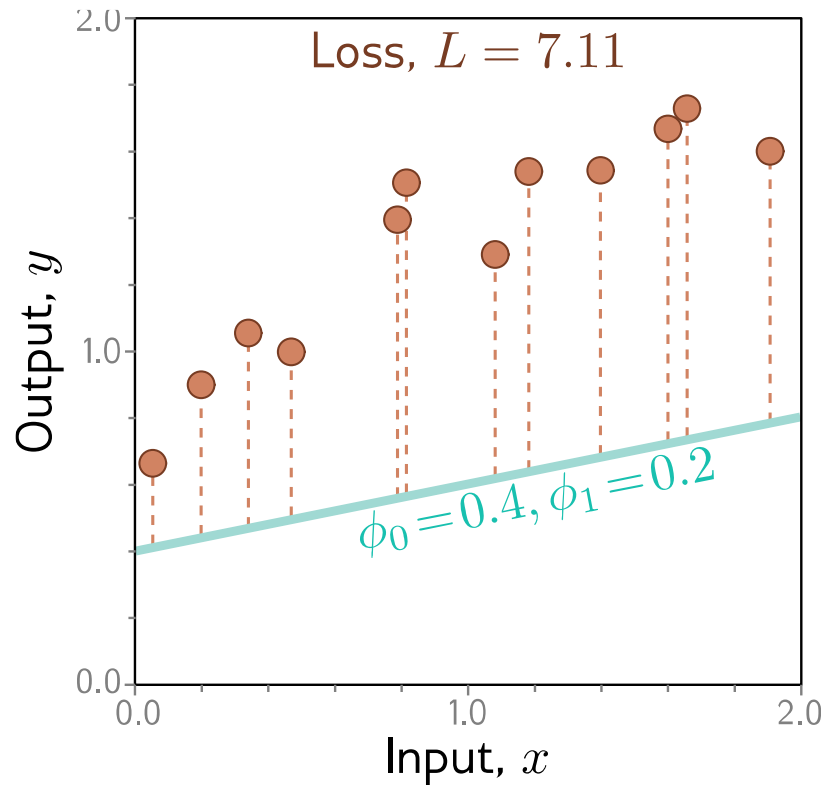


Loss function:

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

“Least squares loss function”

Example: 1D Linear regression loss function

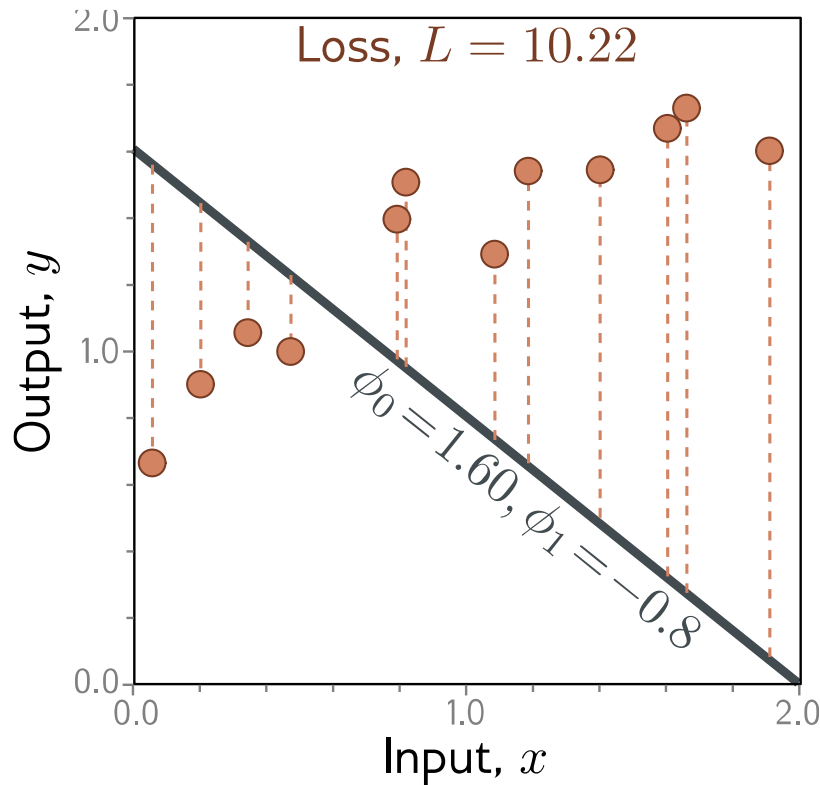


Loss function:

$$L[\phi] = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2$$
$$= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

“Least squares loss function”

Example: 1D Linear regression loss function

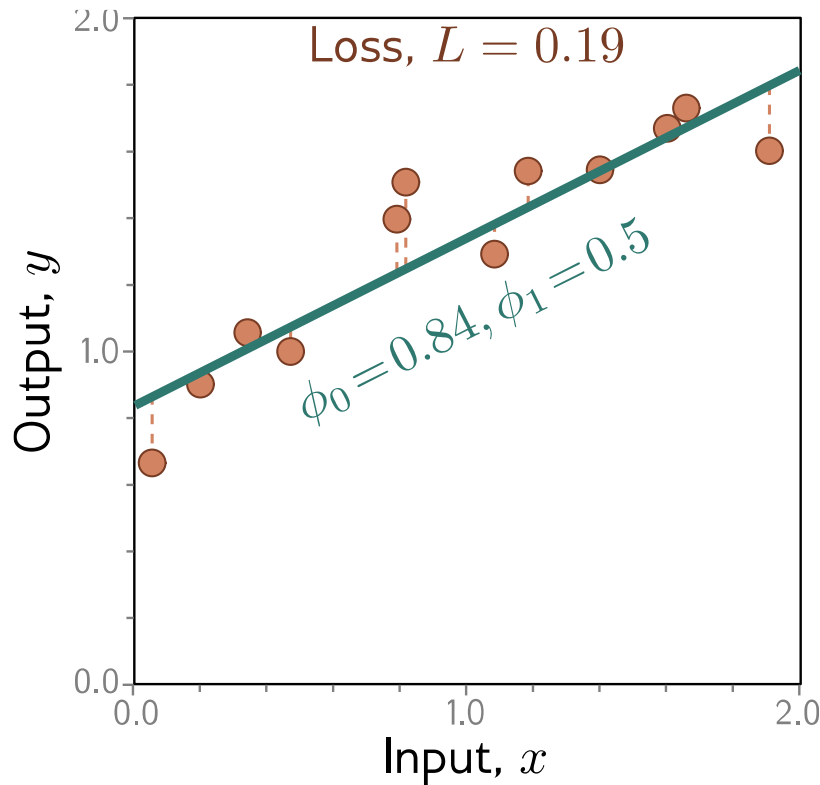


Loss function:

$$L[\phi] = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2$$
$$= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

“Least squares loss function”

Example: 1D Linear regression loss function

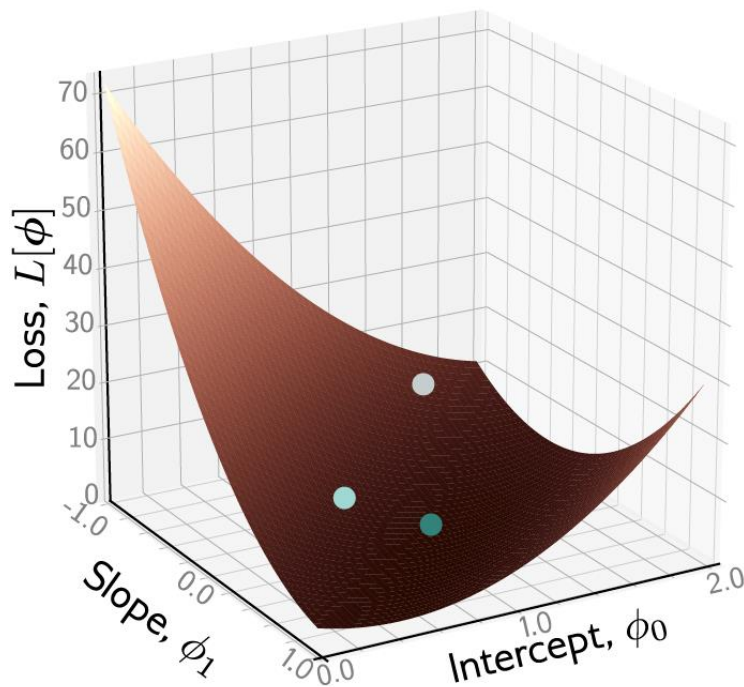


Loss function:

$$L[\phi] = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2$$
$$= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

“Least squares loss function”

Example: 1D Linear regression loss function

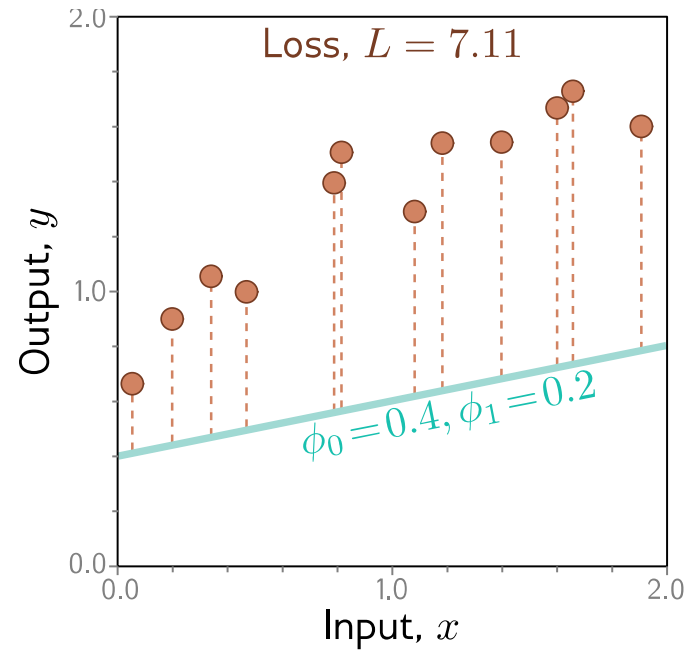
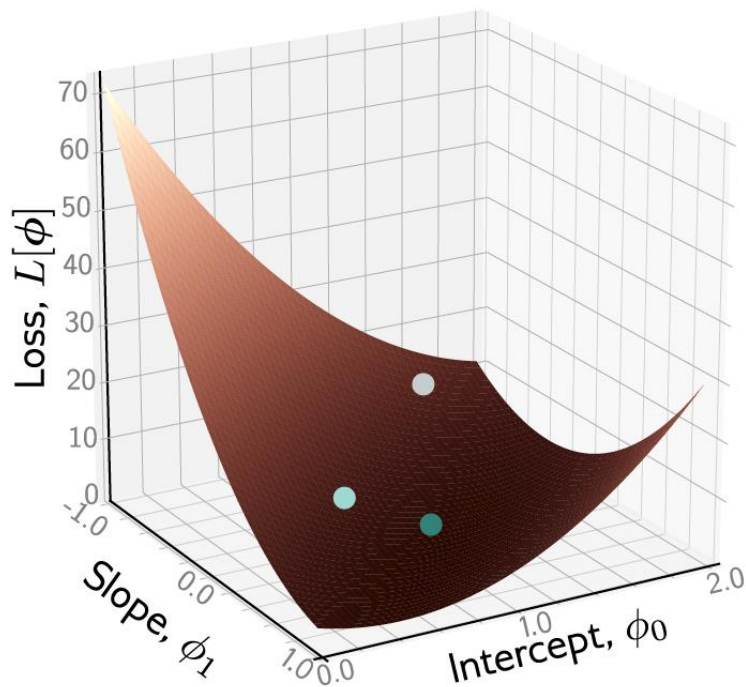


Loss function:

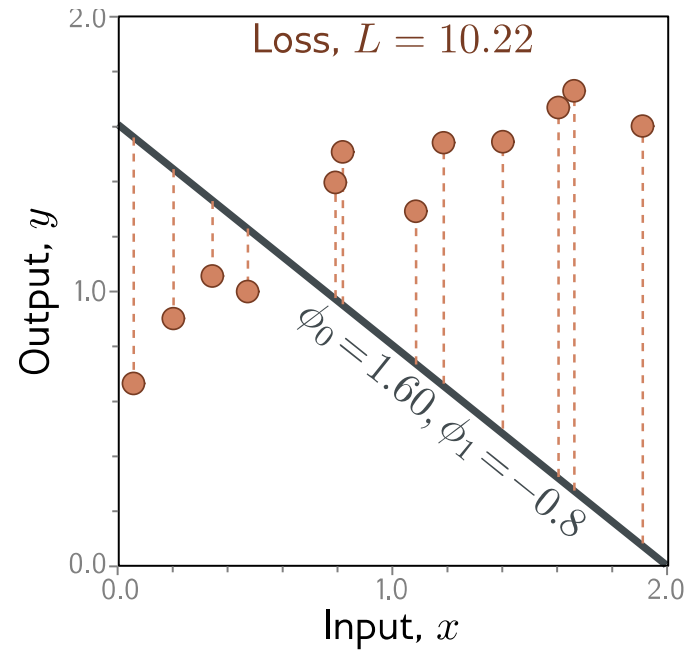
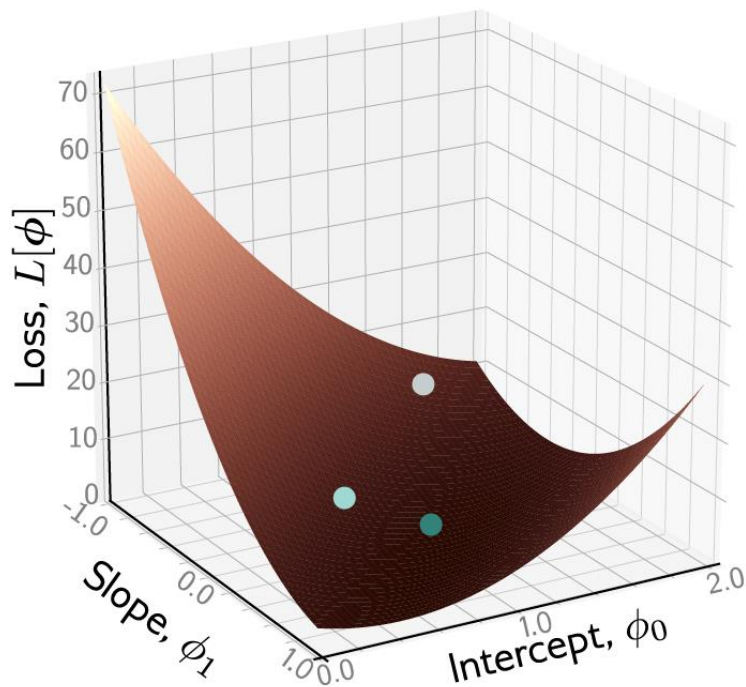
$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

“Least squares loss function”

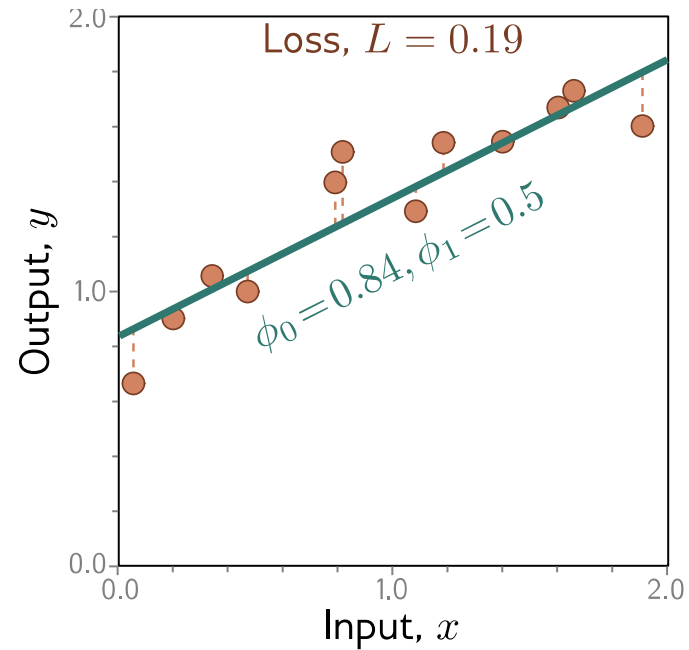
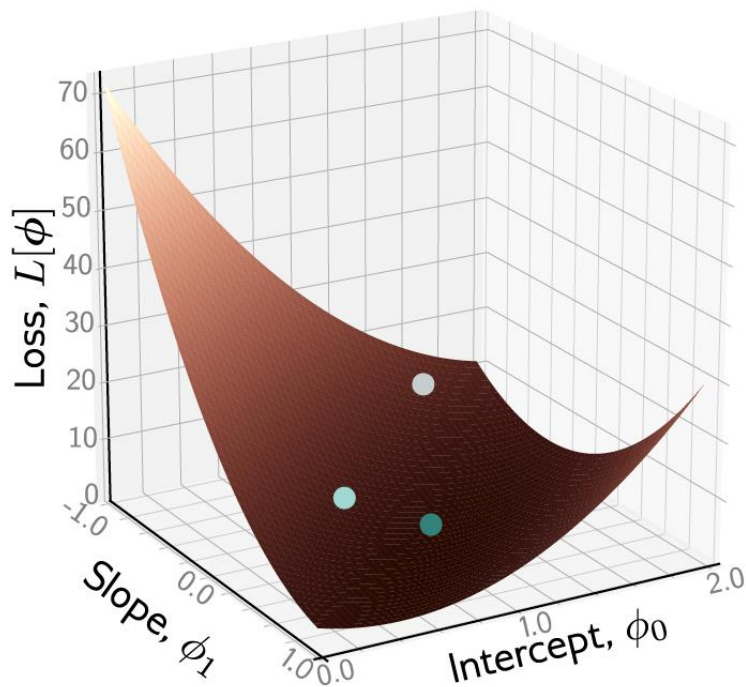
Example: 1D Linear regression loss function



Example: 1D Linear regression loss function

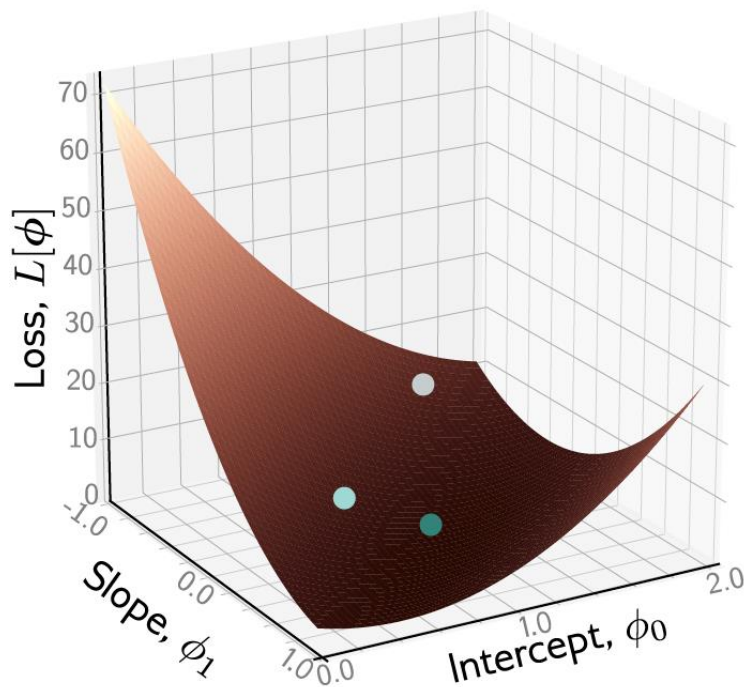


Example: 1D Linear regression loss function

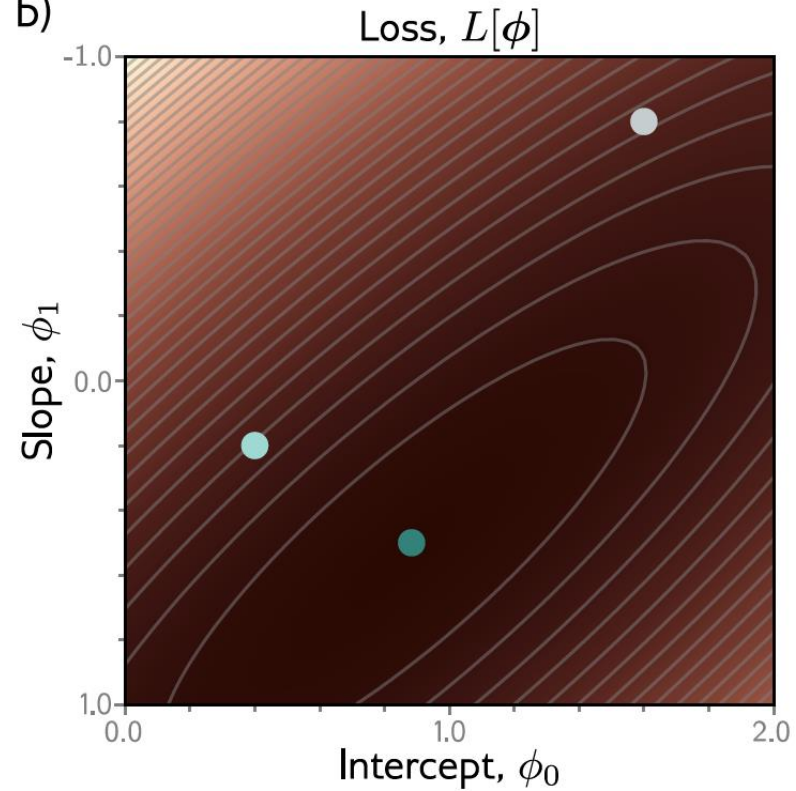


Example: 1D Linear regression loss function

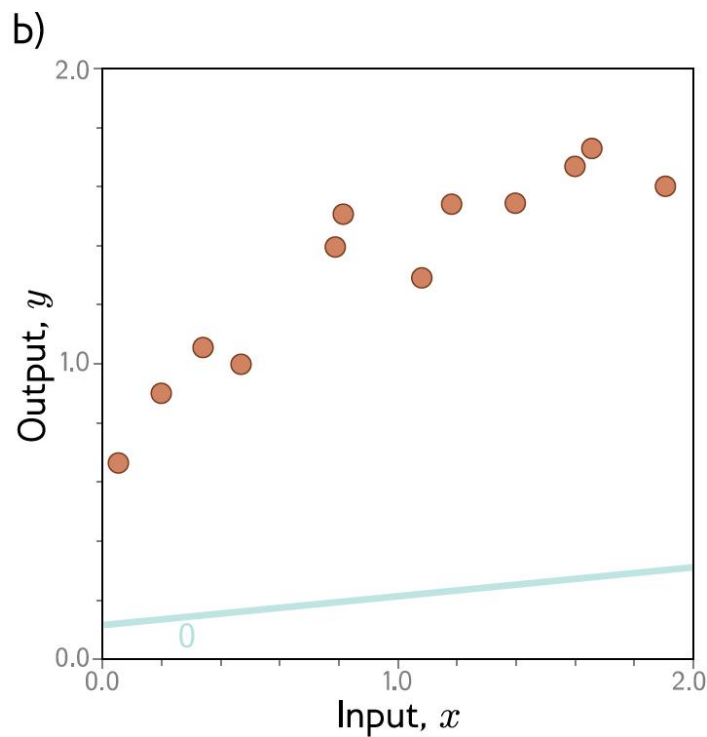
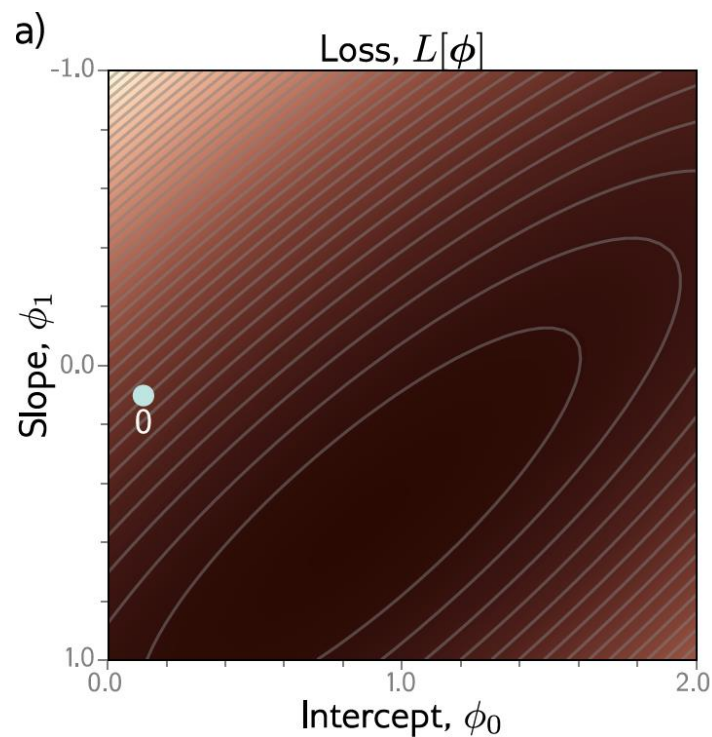
a)



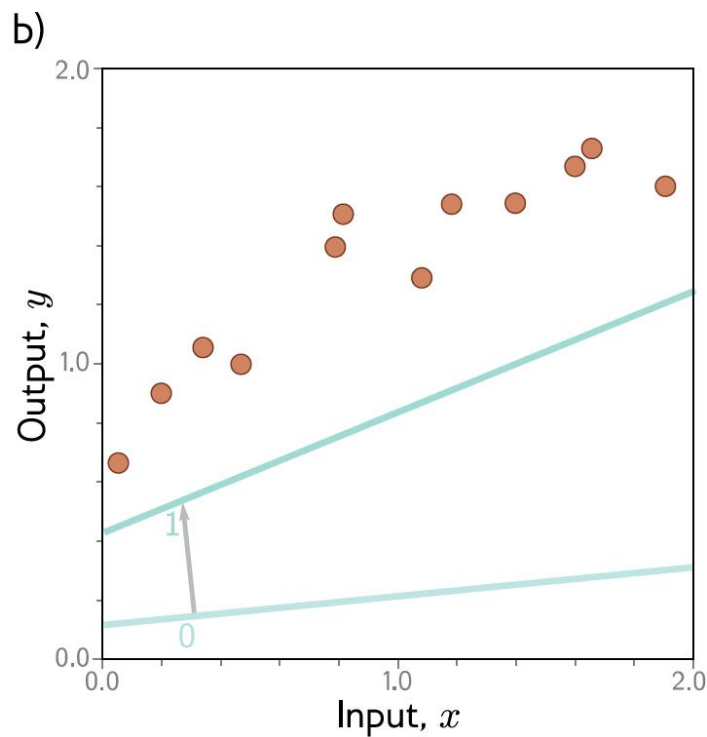
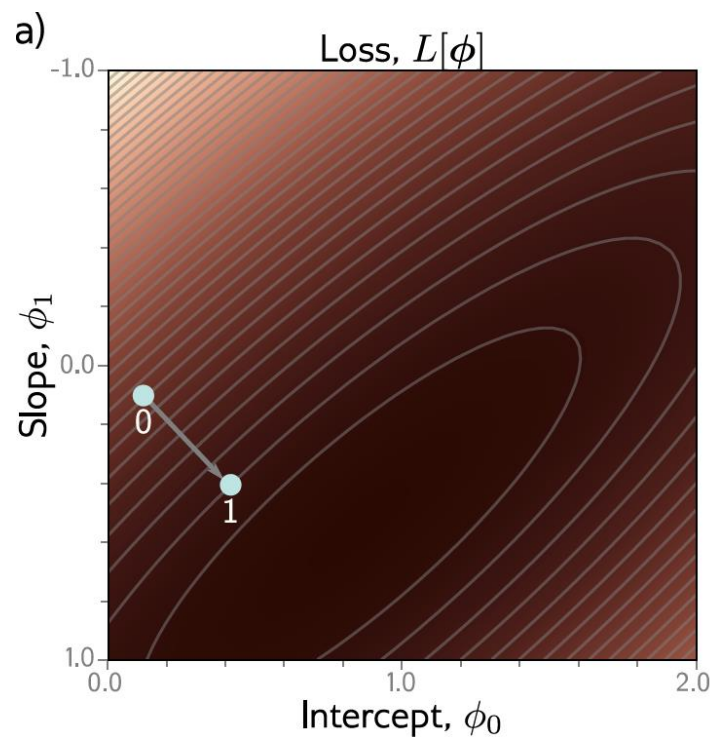
b)



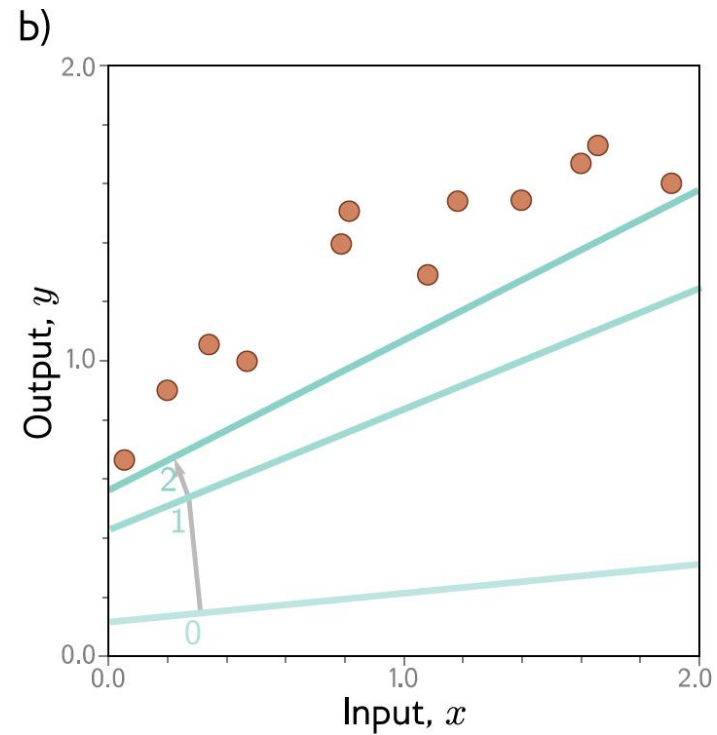
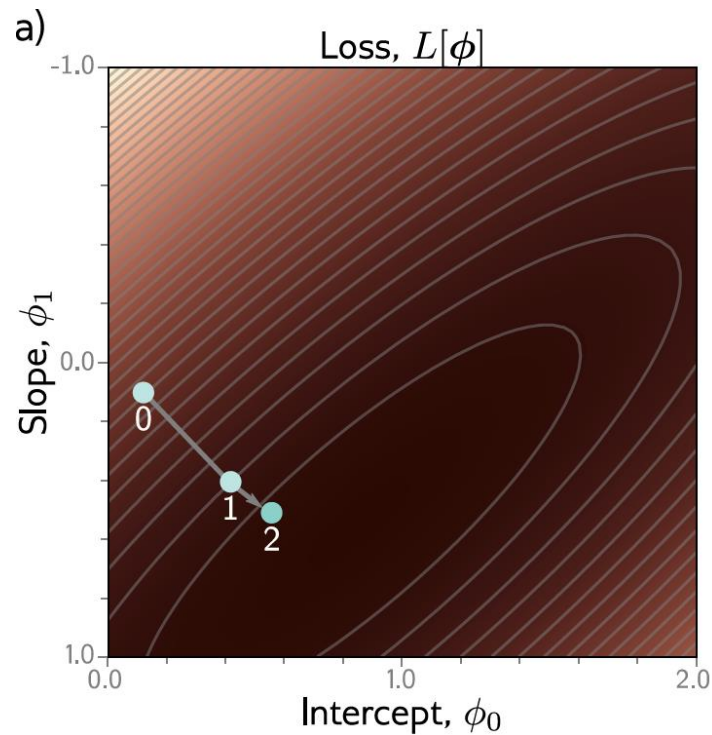
Example: 1D Linear regression training



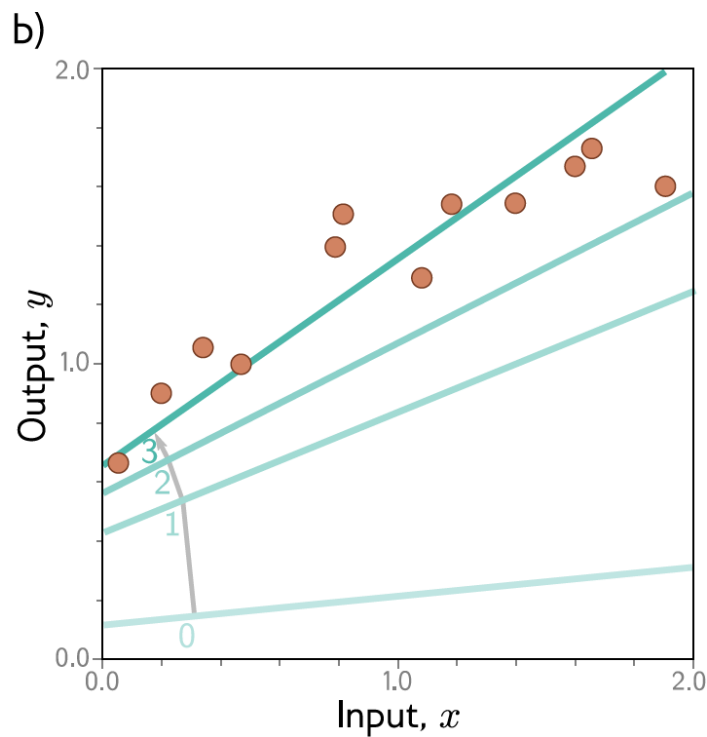
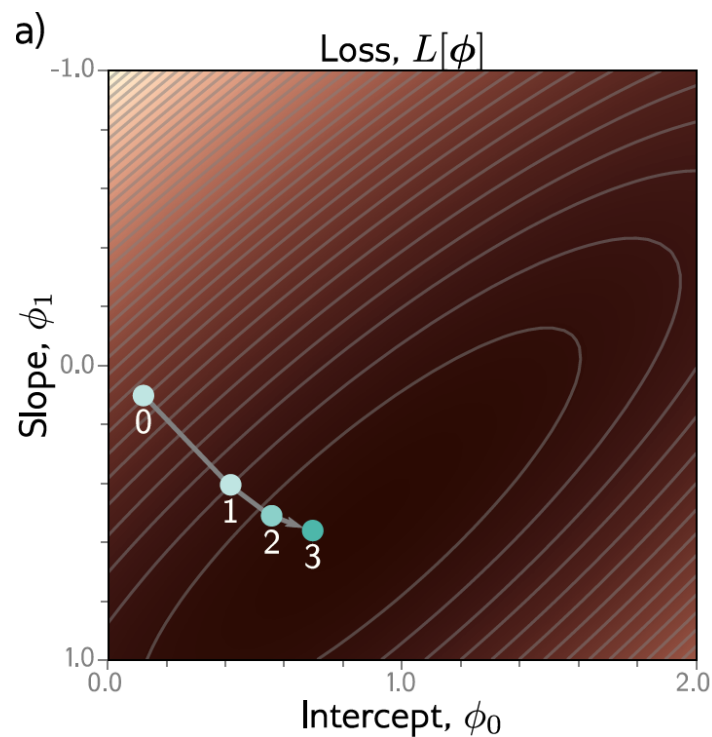
Example: 1D Linear regression training



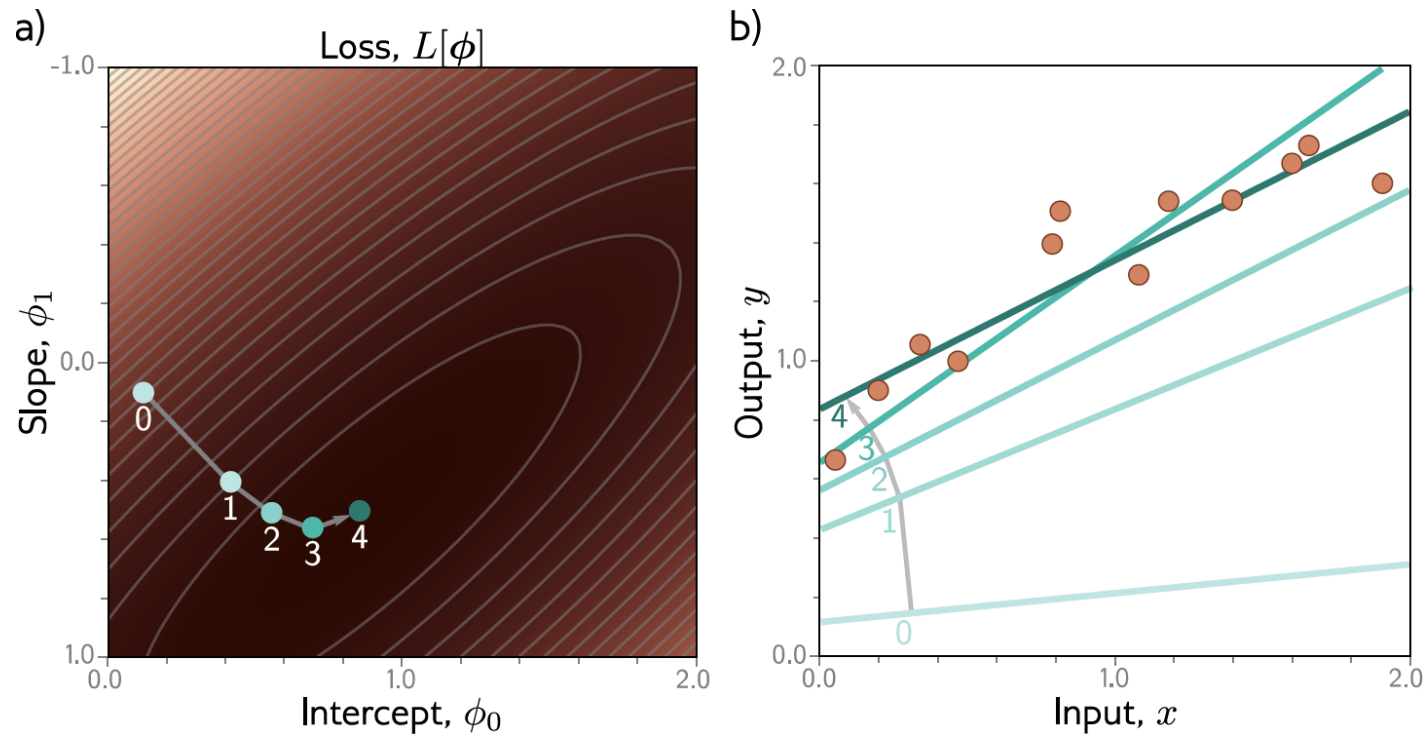
Example: 1D Linear regression training



Example: 1D Linear regression training



Example: 1D Linear regression training



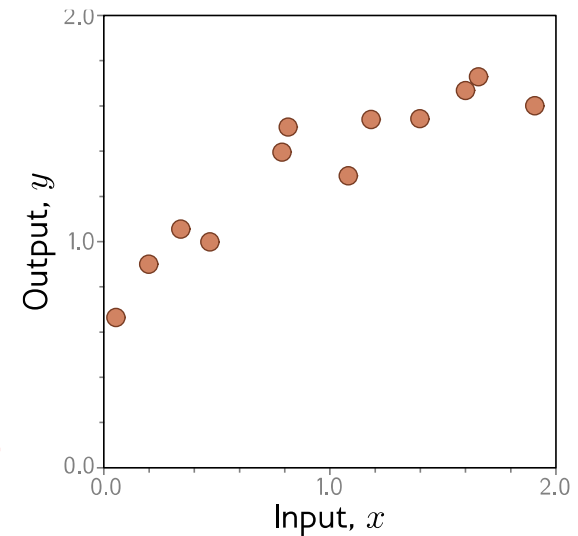
This technique is known as
gradient descent

Possible objections

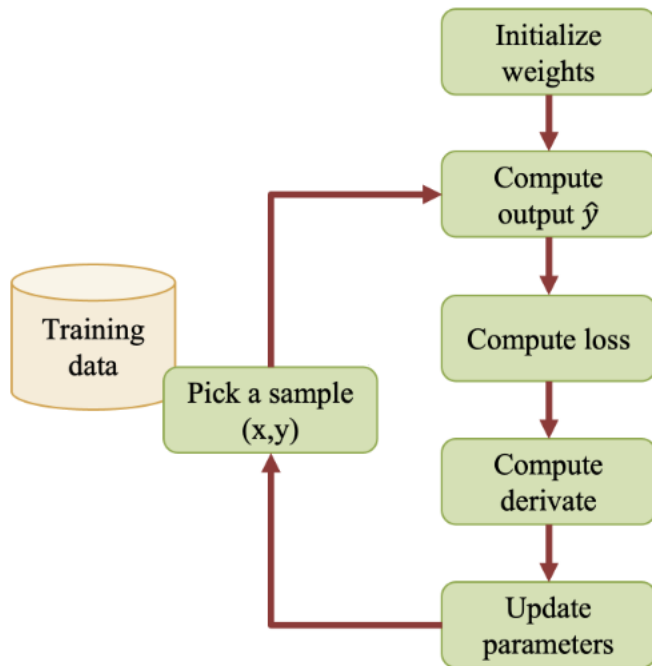
- But you can fit the line model in closed form!
 - Yes – but we won't be able to do this for more complex models
- But we could exhaustively try every slope and intercept combo!
 - Yes – but we won't be able to do this when there are a million parameters

Example: 1D Linear regression testing

- Test with different set of paired input/output data
 - Measure performance
 - Degree to which this is same as training = **generalization**
- Might not generalize well because
 - Model too simple
 - Model too complex
 - fits to statistical peculiarities of data
 - this is known as **overfitting**



Linear regression training



1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

η is learning rate

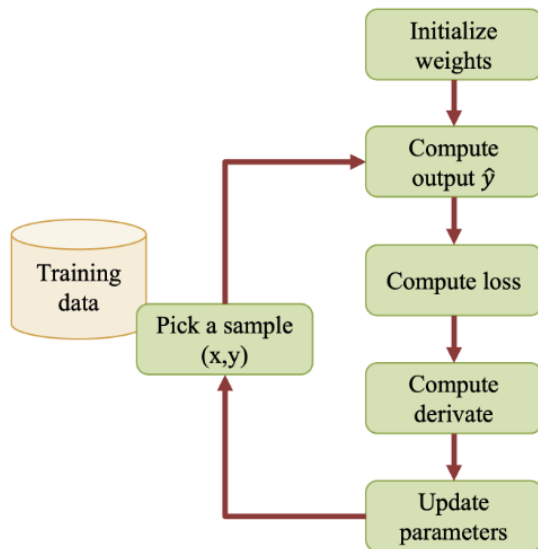
$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

example

File advertising

	A	B	C	D	E
1	TV	Radio	Newspaper	Sales	
2	230.1	37.8	69.2	22.1	
3	44.5	39.3	45.1	10.4	
4	17.2	45.9	69.3	12	
5	151.5	41.3	58.5	16.5	
6	180.8	10.8	58.4	17.9	
7	8.7	48.9	75	7.2	
8	57.5	32.8	23.5	11.8	
9	120.2	19.6	11.6	13.2	
10	8.6	2.1	1	4.8	
11	199.8	2.6	21.2	15.6	
12	66.1	5.8	24.2	12.6	
13	214.7	24	4	17.4	
14	23.8	35.1	65.9	9.2	
15	97.5	7.6	7.2	13.7	
16	204.1	32.9	46	19	
17	195.4	47.7	52.9	22.4	
18	67.8	36.6	114	12.5	
19	281.4	39.6	55.8	24.4	
20	69.2	20.5	18.3	11.3	



1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

η is learning rate

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b.$$

$$L = (\hat{y} - y)^2.$$

$$\frac{\partial L}{\partial w_1} = 2x_1(\hat{y} - y), \quad \frac{\partial L}{\partial w_2} = 2x_2(\hat{y} - y), \quad \frac{\partial L}{\partial w_3} = 2x_3(\hat{y} - y), \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y).$$

$$w_1 \leftarrow w_1 - \eta \frac{\partial L}{\partial w_1}, \quad w_2 \leftarrow w_2 - \eta \frac{\partial L}{\partial w_2}, \quad w_3 \leftarrow w_3 - \eta \frac{\partial L}{\partial w_3}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

```
# dataset
!gdown # Your URL
!unzip advertising.zip

import numpy as np
import matplotlib.pyplot as plt
import random

def get_column(data, index):
    result = [row[index] for row in data]
    return result

def prepare_data(file_name_dataset):
    data = np.genfromtxt(file_name_dataset, delimiter=',', skip_header=1).tolist()
    N = len(data)

    # get tv (index=0)
    tv_data = get_column(data, 0)

    # get radio (index=1)
    radio_data = get_column(data, 1)

    # get newspaper (index=2)
    newspaper_data = get_column(data, 2)

    # get sales (index=3)
    sales_data = get_column(data, 3)

    # building X input and y output for training
    X = [tv_data, radio_data, newspaper_data]
    y = sales_data
    return X,y
```

```
def initialize_params():
    # w1 = random.gauss(mu=0.0, sigma=0.01)
    # w2 = random.gauss(mu=0.0, sigma=0.01)
    # w3 = random.gauss(mu=0.0, sigma=0.01)
    # b = 0

    w1, w2, w3, b = (0.016992259082509283, 0.0070783670518262355, -0.002307860847821344, 0)
    return w1, w2, w3, b

print(initialize_params())

# compute output and loss
def predict(x1, x2, x3, w1, w2, w3, b):
    return w1*x1 + w2*x2 + w3*x3 + b

def compute_loss(y_hat, y):
    return (y_hat - y)**2

# compute gradient
def compute_gradient_wi(xi, y, y_hat):
    dl_dwi = 2*xi*(y_hat-y)
    return dl_dwi

def compute_gradient_b(y, y_hat):
    dl_db = 2*(y_hat-y)
    return dl_db

# update weights
def update_weight_wi(wi, dl_dwi, lr):
    wi = wi - lr*dl_dwi
    return wi

def update_weight_b(b, dl_db, lr):
    b = b - lr*dl_db
    return b
```

```

def implement_linear_regression(X_data, y_data, epoch_max = 50, lr = 1e-5):
    losses = []

    w1, w2, w3, b = initialize_params()

    N = len(y_data)
    for epoch in range(epoch_max):
        for i in range(N):
            # get a sample
            x1 = X_data[0][i]
            x2 = X_data[1][i]
            x3 = X_data[2][i]

            y = y_data[i]

            # print(y)
            # compute output
            y_hat = predict(x1, x2, x3, w1, w2, w3, b)

            # compute loss
            loss = compute_loss(y, y_hat)

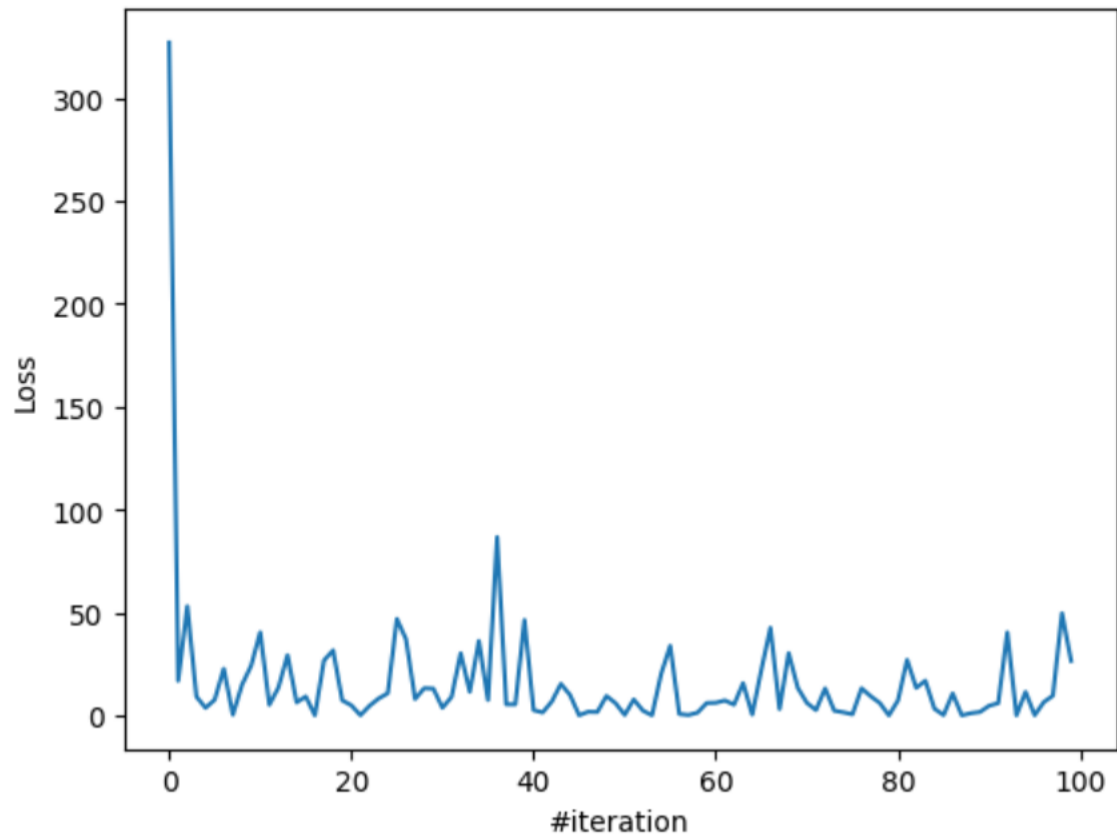
            # compute gradient w1, w2, w3, b
            dl_dw1 = compute_gradient_wi(x1, y, y_hat)
            dl_dw2 = compute_gradient_wi(x2, y, y_hat)
            dl_dw3 = compute_gradient_wi(x3, y, y_hat)
            dl_db = compute_gradient_b(y, y_hat)

            # update parameters
            w1 = update_weight_wi(w1, dl_dw1, lr)
            w2 = update_weight_wi(w2, dl_dw2, lr)
            w3 = update_weight_wi(w3, dl_dw3, lr)
            b = update_weight_b(b, dl_db, lr)

            # logging
            losses.append(loss)
    return (w1, w2, w3, b, losses)

```

```
X,y = prepare_data('../content/advertising.csv')
(w1,w2,w3,b, losses) = implement_linear_regression(X,y)
print(losses[0:100])
plt.plot(losses[0:100])
plt.xlabel("#iteration")
plt.ylabel("Loss")
plt.show()
```



```
X,y = prepare_data('../content/advertising.csv')
(w1,w2,w3,b, losses) = implement_linear_regression(X,y)
print(w1,w2,w3)
```

0.07405984066396477 0.15917360263437663 0.017561197559948935

```
# given new data
tv = 19.2
radio = 35.9
newspaper = 51.3

X,y = prepare_data('../content/advertising.csv')
(w1,w2,w3,b, losses) = implement_linear_regression(X,y)
sales = predict(tv, radio, newspaper, w1, w2, w3, b)
print(f'predicted sales is {sales}')
```

predicted sales is 8.176413319549823

Data Normalization

Min-Max Scaling - > data will be scaled to [0 1]

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Inverse Min-Max Scaling

$$x = x' \times (x_{\max} - x_{\min}) + x_{\min}$$

```
..
# 1. Min-Max Scaling
```

```
# -----
```

```
def min_max_scaling(X):
```

```
    mins = []
```

```
    maxs = []
```

```
    X_scaled = []
```

```
for feature in X:
```

```
    min_val = min(feature)
```

```
    max_val = max(feature)
```

```
    mins.append(min_val)
```

```
    maxs.append(max_val)
```

```
    scaled_feature = []
```

```
    for x in feature:
```

```
        scaled_x = (x - min_val) / (max_val - min_val)
```

```
        scaled_feature.append(scaled_x)
```

```
    X_scaled.append(scaled_feature)
```

```
return X_scaled, mins, maxs
```

2. Inverse Min-Max

```
def inverse_min_max_scaling(X_scaled, mins, maxs):  
    X_recovered = []  
  
    for i in range(len(X_scaled)):  
        feature = X_scaled[i]  
        min_val = mins[i]  
        max_val = maxs[i]  
  
        recovered_feature = []  
        for x in feature:  
            original_x = x * (max_val - min_val) + min_val  
            recovered_feature.append(original_x)  
  
        X_recovered.append(recovered_feature)  
  
    return X_recovered
```

Z-score Scaling (Standardization)

$$x' = \frac{x - \mu}{\sigma}$$

→ New data: mean = 0 and variance = 1

Inverse Z-score Scaling

$$x = x' \times \sigma + \mu$$

3. Z-Score Scaling

```
def z_score_scaling(X):
```

```
    means = []
```

```
    stds = []
```

```
    X_scaled = []
```

```
    for feature in X:
```

```
        mean_val = sum(feature) / len(feature)
```

```
        means.append(mean_val)
```

```
    std_val = (sum((x - mean_val) ** 2 for x in feature) / (len(feature) - 1)) ** 0.5
```

```
    stds.append(std_val)
```

```
    scaled_feature = []
```

```
    for x in feature:
```

```
        scaled_x = (x - mean_val) / std_val
```

```
        scaled_feature.append(scaled_x)
```

```
    X_scaled.append(scaled_feature)
```

```
    return X_scaled, means, stds
```

```
# 4. Inverse Z-Score
```

```
# -----
```

```
def inverse_z_score_scaling(X_scaled, means, stds):
```

```
    X_recovered = []
```

```
    for i in range(len(X_scaled)):
```

```
        feature = X_scaled[i]
```

```
        mean_val = means[i]
```

```
        std_val = stds[i]
```

```
        recovered_feature = []
```

```
        for x in feature:
```

```
            original_x = x * std_val + mean_val
```

```
            recovered_feature.append(original_x)
```

```
        X_recovered.append(recovered_feature)
```

```
    return X_recovered
```